



PROGRAMAÇÃO WEB 2 · AULA 06

A API de login

Quem confere a senha agora é o servidor.

Emanuel Nascente · Escola Técnica Mesquita

FASTAPI



0 que vamos fazer hoje

01 Entender as peças: **cliente, servidor, API, HTTP e JSON**.

02 Entender as ferramentas: **FastAPI, venv e uv** — e para que serve cada uma.

03 Construir uma API em Python com **uma rota só: POST /login**.

04 Testar tudo no navegador, pelo **Swagger** que o FastAPI gera sozinho.

0 furo da aula 5

Quem confere a senha **não pode** ser o navegador do visitante.

Na aula passada, bastou digitar `localStorage.setItem("logado", "true")` no console para entrar sem senha.

E a senha certa está escrita no Login.jsx, que chega inteiro ao navegador: qualquer um lê.

A saída: levar essa decisão para o servidor, onde o visitante não mexe.

A senha
escrita no Login.jsx

+

A sessão
localStorage

=

Qualquer um
lê e forja

Dois programas conversando

Cliente

React · navegador

Desenha a tela e recolhe o que a pessoa digitou. Roda no computador de quem usa — portanto não é confiável.

PEDIDO

me deixa entrar?
e-mail + senha

RESPOSTA

pode entrar ou não pode

Servidor

Python · FastAPI

Guarda as regras e os dados. Roda numa máquina nossa, que o visitante não enxerga por dentro.

O cliente nunca decide sozinho: ele pergunta. Quem decide é o servidor, e a resposta dele é o que vale.

Até hoje nosso sistema só tinha o lado esquerdo. Hoje nasce o direito.

O que é uma API

API é o **cardápio** do servidor: os pedidos que ele aceita.

API vem de Application Programming Interface: interface de programação. Não é uma tela para gente — é uma porta para outros programas.

Pense num restaurante: o cliente não entra na cozinha. Ele pede ao garçom um prato **que está no cardápio**, e o garçom traz o prato pronto.

Nossa API vai ter um único item no cardápio: o login. Cada item desses se chama rota (ou endpoint).

O pedido

`POST /login`

+

A API confere

`main.py`

=

A resposta

`200` ou `401`

Como é um pedido de verdade

Pedido

request

Método (o verbo), endereço (a rota) e, quando precisa mandar dados, um corpo.

O REACT MANDA

```
POST /login
{"email": "...", "senha": "..."}

```

A API DEVOLVE

```
200 OK
{"mensagem": "Login realizado..."}

```

Resposta

response

Um código de status, que diz como foi, e um corpo com o resultado.

HTTP é o idioma dessa conversa — o mesmo que o navegador usa para abrir qualquer site. **GET** busca algo; **POST** envia dados para o servidor processar.

Login é POST: a senha vai no **corpo** do pedido, e não pendurada no endereço, onde ficaria no histórico do navegador.

O código de status diz como foi

200 **OK** — deu certo. Todo pedido volta com um número desses. Aqui: e-mail e senha conferem.

401 **Unauthorized** — não autorizado. Aqui: e-mail ou senha errados.

404 **Not Found** — essa rota não existe. É o que aparece ao abrir a raiz /.

422 **Unprocessable Entity** — o corpo veio no formato errado, faltando um campo, por exemplo.

JSON: o formato da conversa

pedido · o React manda (JSON)

```
1  {  
2    "email": "user@email.com",  
3    "senha": "123456"  
4  }
```

resposta · o main.py devolve (dicionário)

```
1  {  
2    "mensagem": "Login realizado com sucesso!",  
3    "email": dados.email,  
4  }
```

JSON é texto com chaves e valores. O React fala JavaScript, a API fala Python — e os dois sabem ler e escrever **JSON**.

Em Python, a estrutura parecida se chama **dicionário**. O FastAPI converte sozinho: o dicionário que a rota devolve sai como JSON, e o JSON que chega vira um objeto Python.

FastAPI e quem vem junto

01

FastAPI

O framework: escrevemos funções Python e ele as transforma em rotas HTTP. O Pydantic, que vem junto, confere o corpo do pedido.

```
$ @app.post("/login")
```

Framework = o esqueleto pronto; a gente preenche.

02

Uvicorn

O servidor: o programa que fica ligado escutando a porta 8000 e entrega cada pedido que chega para o FastAPI. Sem ele o código não atende ninguém.

```
$ fastapi dev main.py
```

O fastapi dev liga o Uvicorn por nós.

03

Swagger

A documentação automática. O FastAPI lê as rotas e monta uma página onde dá para ver e testar cada uma, sem escrever uma linha de front-end.

```
$ http://127.0.0.1:8000/docs
```

Segue o padrão OpenAPI; /redoc mostra outra versão.

O problema das dependências

Um Python só para todos os projetos **não dá certo**.

Dependência é um pacote que o projeto usa e que não veio com o Python: o FastAPI é uma. O instalador de pacotes padrão do Python chama pip.

Instalando tudo no Python do computador, os projetos passam a dividir a mesma estante: o de hoje quer a versão 0.141, um antigo quer a 0.100 — e só cabe uma.

Outro problema: o colega clona o projeto e não tem como saber quais pacotes (e quais versões) instalar.

Projeto antigo

`fastapi 0.100`

+

Projeto novo

`fastapi 0.141`

=

Python do computador

só cabe um

venv: um Python só deste projeto

A pasta **.venv** é a estante particular do projeto.

venv (virtual environment, ambiente virtual) é uma pasta dentro do projeto com um Python e os pacotes **só dele**. Cada projeto tem a sua; ninguém briga.

Ativar o venv é avisar ao terminal: “daqui para frente, python e fastapi são os desta pasta”. O sinal é o nome do projeto entre parênteses no começo da linha.

A .venv não vai para o GitHub: é pesada e se recria em segundos.

Python
3.12
+
Pacotes
fastapi, pydantic...
=

Ambiente do projeto

api-login/.venv

uv: uma ferramenta para tudo isso

01

Instala o Python

Pedimos a versão 3.12 e o uv resolve: usa a que já existe na máquina ou baixa uma, sem mexer no Python que você já tem.

```
$ uv init api-login --python 3.12
```

02

Cria o venv

Um comando cria a pasta .venv com o Python certo dentro. É o mesmo venv de sempre do Python — o uv só faz mais rápido.

```
$ uv venv
```

03

Cuida dos pacotes

Instala no venv, anota no pyproject.toml e grava as versões exatas no uv.lock — assim o colega instala igualzinho.

```
$ uv add "fastapi[standard]"  
$ uv sync
```

Faz o trabalho do pip, só que muito mais rápido.

Vocês já conhecem isso: é o npm do Python

O QUÊ	NO REACT (NPM)	NO PYTHON (UV)
A ficha do projeto	<code>package.json</code>	<code>pyproject.toml</code>
As versões exatas	<code>package-lock.json</code>	<code>uv.lock</code>
Onde os pacotes moram	<code>node_modules/</code>	<code>.venv/</code>
Adicionar um pacote	<code>npm install react-router-dom</code>	<code>uv add "fastapi[standard]"</code>
Instalar o que a ficha pede	<code>npm install</code>	<code>uv sync</code>
Rodar em desenvolvimento	<code>npm run dev</code>	<code>fastapi dev main.py</code>
Endereço de desenvolvimento	<code>localhost:5173</code>	<code>127.0.0.1:8000</code>

A diferença: o `node_modules` isola sozinho; no Python o `venv` precisa estar **ativo**.

Preparar o ambiente

A Crie a pasta **C:\projetos** e abra no VS Code: **File** → **Open Folder** (em português: Arquivo → Abrir Pasta).

B Abra o terminal: **Terminal** → **New Terminal** (Terminal → Novo Terminal). No canto do painel tem que aparecer **powershell**.

C Nos slides, digite **só o que vem depois do >**. O **PS C:\projetos>** quem escreve é o próprio terminal.

D Um comando por slide: instalar o uv, criar o projeto e o **.venv**, ativar e adicionar o FastAPI. Compare a saída antes de seguir.

Instalando o uv

```
Terminal · PowerShell

1 PS C:\projetos> powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
2 Downloading uv (x86_64-pc-windows-msvc)
3 Installing to C:\Users\aluno\.local\bin
4   uv.exe
5   uvx.exe
6 everything's installed!
```

Digite (ou copie) só a parte depois de **PS C:\projetos>** e aperte Enter. É uma linha só, e **só se faz uma vez** por computador.

O instalador coloca o uv numa pasta do usuário e a adiciona ao PATH — a lista de lugares onde o terminal procura programas.

A versão e os detalhes da saída podem variar; o que importa é o everything's installed! no fim.

Agora **feche o VS Code inteiro** e abra de novo. O terminal só enxerga o PATH novo quando nasce depois da instalação.

Conferindo a instalação

```
Terminal · PowerShell

1 PS C:\projetos> uv --version
2 uv 0.6.10 (f2a2d982b 2025-03-25)
```

Se aparecer uv seguido de um número de versão, está pronto. O número no seu computador provavelmente vai ser outro — **tudo bem**.

Não precisa instalar o Python antes: o uv cuida disso no próximo passo.

Deu **O termo 'uv' não é reconhecido...**? O terminal é anterior à instalação. Feche o VS Code inteiro, abra de novo e repita.

Criando o projeto

```
Terminal · PowerShell

1 PS C:\projetos> uv init api-login --python 3.12
2 Initialized project `api-login` at `C:\projetos\api-login`
3 PS C:\projetos> cd api-login
4 PS C:\projetos\api-login>
```

uv init cria a pasta **api-login** com o esqueleto de um projeto Python. O **--python 3.12** fixa a versão do Python — todo mundo da turma usa a mesma.

Depois, **cd api-login** entra na pasta. Todos os comandos seguintes são dados **de dentro dela**.

Confira o fim da linha do terminal: tem que terminar em **\api-login>**. Se não terminar, os próximos passos falham.

O que o uv init criou

```
Explorer · api-login

1  api-login/
2  |— .git/           # o repositório do Git
3  |— .gitignore      # o que o Git não guarda
4  |— .python-version # a versão do Python: 3.12
5  |— main.py         # o nosso código
6  |— pyproject.toml  # a ficha do projeto
7  |— README.md       # a descrição do projeto
```

pyproject.toml é a ficha do projeto: nome, versão do Python e dependências — o package.json do Python.

.python-version diz ao uv qual Python usar neste projeto.

main.py veio com um “Hello” de exemplo. Vamos trocar tudo por nossa API.

.git e .gitignore só aparecem se o Git estiver instalado.

Criando o ambiente virtual

```
Terminal · PowerShell

1 PS C:\projetos\api-login> uv venv
2 Using CPython 3.12.9
3 Creating virtual environment at: .venv
4 Activate with: .venv\Scripts\activate
```

uv venv cria a pasta **.venv** dentro do projeto, com o Python 3.12. Se a máquina não tiver essa versão, o uv baixa antes — aparece uma linha Downloading a mais. Repare na última linha: o próprio uv já ensina o comando do próximo slide.

Por enquanto a **.venv** tem só o Python. Os pacotes entram daqui a dois slides, com o **uv add**.

Ativando o venv

```
Terminal · PowerShell

1 PS C:\projetos\api-login> .venv\Scripts\activate
2 (api-login) PS C:\projetos\api-login>
3
4 # no Prompt de Comando (cmd), o mesmo comando:
5 C:\projetos\api-login> .venv\Scripts\activate
6 # no Git Bash:
7 $ source .venv/Scripts/activate
```

Deu certo quando aparece **(api-login)** no começo da linha. A partir daí, python e fastapi são os da .venv.

A ativação vale só para aquele terminal: abriu outro, ative de novo.

Erro **a execução de scripts foi desabilitada neste sistema?** Rode uma vez **Set-ExecutionPolicy -Scope CurrentUser RemoteSigned**, responda S e ative de novo.

Adicionando o FastAPI

```
Terminal · PowerShell

1 (api-login) PS C:\projetos\api-login> uv add "fastapi[standard]==0.141.1"
2 Resolved 46 packages in 2.01s
3 Installed 44 packages in 5.23s
4 + fastapi==0.141.1
5 + pydantic==2.13.5
6 + uvicorn==0.53.0
7 ... e mais 41 pacotes
```

[standard] pede o FastAPI com os extras: o Uvicorn e o comando fastapi.

==0.141.1 fixa a versão — a mesma para toda a turma.

As aspas são obrigatórias: sem elas o terminal pode entender os colchetes do jeito dele.

São ~44 pacotes porque o FastAPI também tem as dependências dele. O uv resolve a árvore inteira sozinho.

O que o uv add mudou

```
pyproject.toml
1  [project]
2  name = "api-login"
3  version = "0.1.0"
4  description = "Add your description here"
5  readme = "README.md"
6  requires-python = ">=3.12"
7  dependencies = [
8      "fastapi[standard]==0.141.1",
9  ]
```

Três coisas, e nenhuma delas a gente edita à mão:

pyproject.toml ganhou o FastAPI na lista de dependências.

uv.lock apareceu com a versão exata de cada um dos 44 pacotes.

.venv recebeu os pacotes instalados de fato.

Para instalar ou remover pacote, use sempre **uv add** e **uv remove**.

Escrever a API

Um arquivo só. No Explorer do VS Code, abra api-login → main.py, apague tudo o que o uv init deixou nele e vá digitando pedaço por pedaço. Salve sempre com Ctrl+S.

A Importar o FastAPI e criar a **app**.

B Descrever o corpo do pedido com uma classe **BaseModel**.

C Criar a rota **POST /login**: responder 200 se bater e 401 se não bater.

Como ler os painéis

exemplo · não digite este

```
1 # ...
2 def login(dados: DadosLogin):
3     email_certo = dados.email == "user@email.com"
4     senha_certa = dados.senha == "123456"
5
6     if email_certo and senha_certa:
7         # o dicionario vira JSON, status 200
8         return {
9     # ...
```

Faixa roxa: é o que você digita **agora**.

Linhas apagadas: já estão no seu arquivo, dos slides anteriores. Aparecem só para mostrar **onde** escrever. Não digite de novo.

...: um pedaço do arquivo que não cabe no slide. Não digite.

Trecho digitado duas vezes: sem erro nenhum, a API aceita **qualquer** senha.

FastAPI()

```
main.py
1  from fastapi import FastAPI, HTTPException
2  from pydantic import BaseModel
3
4  # a aplicacao: as rotas sao penduradas nela
5  app = FastAPI(title="API do Ponto Eletrônico")
6
7  # ...
```

from ... import é o import do Python: traz o FastAPI e o HTTPException do pacote fastapi, e o BaseModel do pydantic.

app é a aplicação. Todas as rotas vão ser penduradas nela.

O **title** é o nome que aparece no topo do Swagger.

Linha que começa com **#** é comentário: o Python ignora.

BaseModel

```
main.py · digite só a faixa destacada

1  # ...
2
3  # o formato do corpo que o front-end manda
4  class DadosLogin(BaseModel):
5      email: str
6      senha: str
7
8  # ...
```

DadosLogin descreve o corpo que o pedido precisa ter: um email e uma senha, os dois texto (**str**).

Herdar de **BaseModel** liga o Pydantic: se faltar um campo, a API recusa com 422 antes de rodar o nosso código.

E o Swagger usa essa classe para montar o exemplo de corpo.

Em Python, a indentação faz parte do código: os campos ficam **4 espaços** para dentro da classe.

@app.post("/login")

```
main.py · digite só a faixa destacada

1  # ...
2
3  # POST /login: confere o e-mail e a senha
4  @app.post("/login")
5  def login(dados: DadosLogin):
6      email_certo = dados.email == "user@email.com"
7      senha_certa = dados.senha == "123456"
8
9  # ...
```

O **@app.post** logo acima da função é um decorador: ele pendura a função na app como a rota **POST /login**.

dados: DadosLogin diz ao FastAPI para ler o corpo do pedido e entregar já no formato da classe.

Cada **==** compara e guarda True ou False.

Não esqueça os **dois-pontos** no fim da linha do def.

return

```
main.py · digite só a faixa destacada

1  # ...
2  def login(dados: DadosLogin):
3      email_certo = dados.email == "user@email.com"
4      senha_certa = dados.senha == "123456"
5
6      if email_certo and senha_certa:
7          # o dicionario vira JSON, status 200
8          return {
9              "mensagem": "Login realizado com sucesso!",
10             "email": dados.email,
11         }
12
13  # ...
```

Se as duas comparações deram True, a função **devolve um dicionário**.

O FastAPI transforma o dicionário em JSON e responde com status **200** — sem a gente escrever nada sobre isso.

O return encerra a função: o que vem depois não roda.

Compare com a aula 4: é a mesma conferência do handleSubmit — só mudou de lado.

HTTPException

```
main.py · digite só a faixa destacada

1  # ...
2      if email_certo and senha_certa:
3          # o dicionario vira JSON, status 200
4          return {
5              "mensagem": "Login realizado com sucesso!",
6              "email": dados.email,
7          }
8
9      # chegou aqui: algo nao bateu, status 401
10     raise HTTPException(
11         status_code=401,
12         detail="E-mail ou senha incorretos.",
13     )
```

Se o código chegou aqui, o if não entrou: algo não bateu.

raise HTTPException interrompe a função e responde na hora com o status **401** e o texto do detail.

A mensagem não diz se o erro foi no e-mail ou na senha — de propósito: não damos pistas para quem tenta adivinhar.

O raise fica no nível do if (4 espaços), **fora** dele.

main.py inteiro

main.py (1)

```
1  from fastapi import FastAPI, HTTPException
2  from pydantic import BaseModel
3
4  # a aplicacao: as rotas sao penduradas nela
5  app = FastAPI(title="API do Ponto Eletrônico")
6
7
8  # o formato do corpo que o front-end manda
9  class DadosLogin(BaseModel):
10     email: str
11     senha: str
12
13
```

main.py (2) continuação

```
14  # POST /login: confere o e-mail e a senha
15  @app.post("/login")
16  def login(dados: DadosLogin):
17     email_certo = dados.email == "user@email.com"
18     senha_certa = dados.senha == "123456"
19
20     if email_certo and senha_certa:
21         # o dicionario vira JSON, status 200
22         return {
23             "mensagem": "Login realizado com sucesso!",
24             "email": dados.email,
25         }
26
27     # chegou aqui: algo nao bateu, status 401
28     raise HTTPException(
29         status_code=401,
30         detail="E-mail ou senha incorretos.",
31     )
```

Confira linha a linha, indentação inclusive. **Nada a mais:** um def login só. Depois, salve (Ctrl+S).

Rodar e testar no Swagger

O terminal fica ocupado com o servidor ligado. Não feche: se fechar, a API desliga.

A

Subir o servidor com `fastapi dev main.py`.

B

Abrir `http://127.0.0.1:8000/docs` no navegador.

C

Testar três pedidos: senha certa, senha errada e corpo incompleto.

Ligando o servidor

```
Terminal · PowerShell

1  (api-login) PS C:\projetos\api-login> fastapi dev main.py
2      Starting FastAPI in development mode
3      Server started at http://127.0.0.1:8000
4      Documentation at http://127.0.0.1:8000/docs
5  INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
6  INFO:      Application startup complete.
```

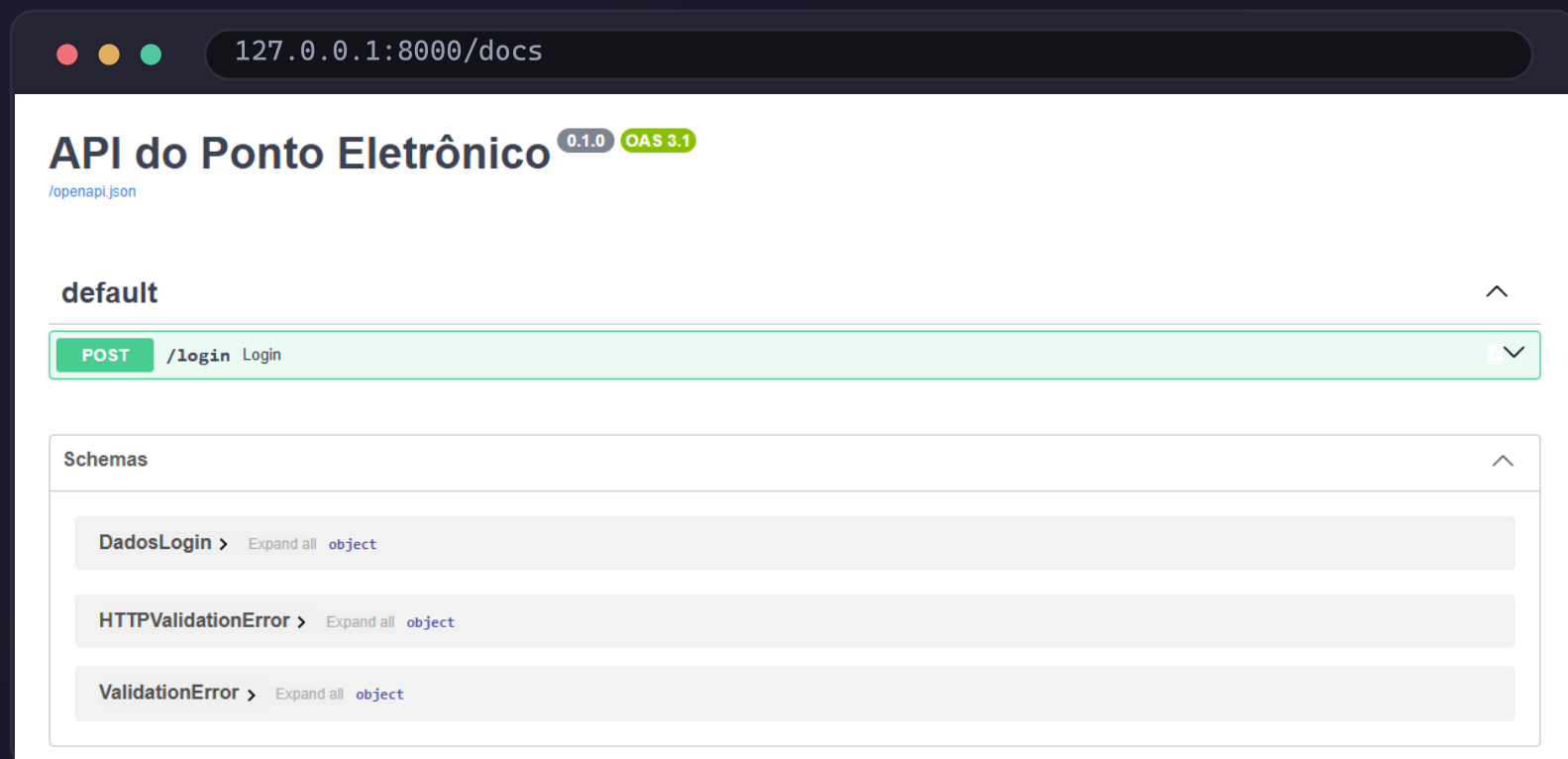
fastapi dev liga o Uvicorn apontando para a app do main.py. As linhas Server started e Documentation trazem os dois endereços.

Segure **Ctrl** e clique no endereço do /docs para abrir no navegador. A saída real tem mais algumas linhas — tudo bem.

Modo dev: salvou o main.py, o servidor reinicia sozinho. Para desligar, **Ctrl+C** no terminal.

127.0.0.1 é o próprio computador (o mesmo que localhost) e **8000** é a porta. O Vite do React usava a 5173.

http://127.0.0.1:8000/docs



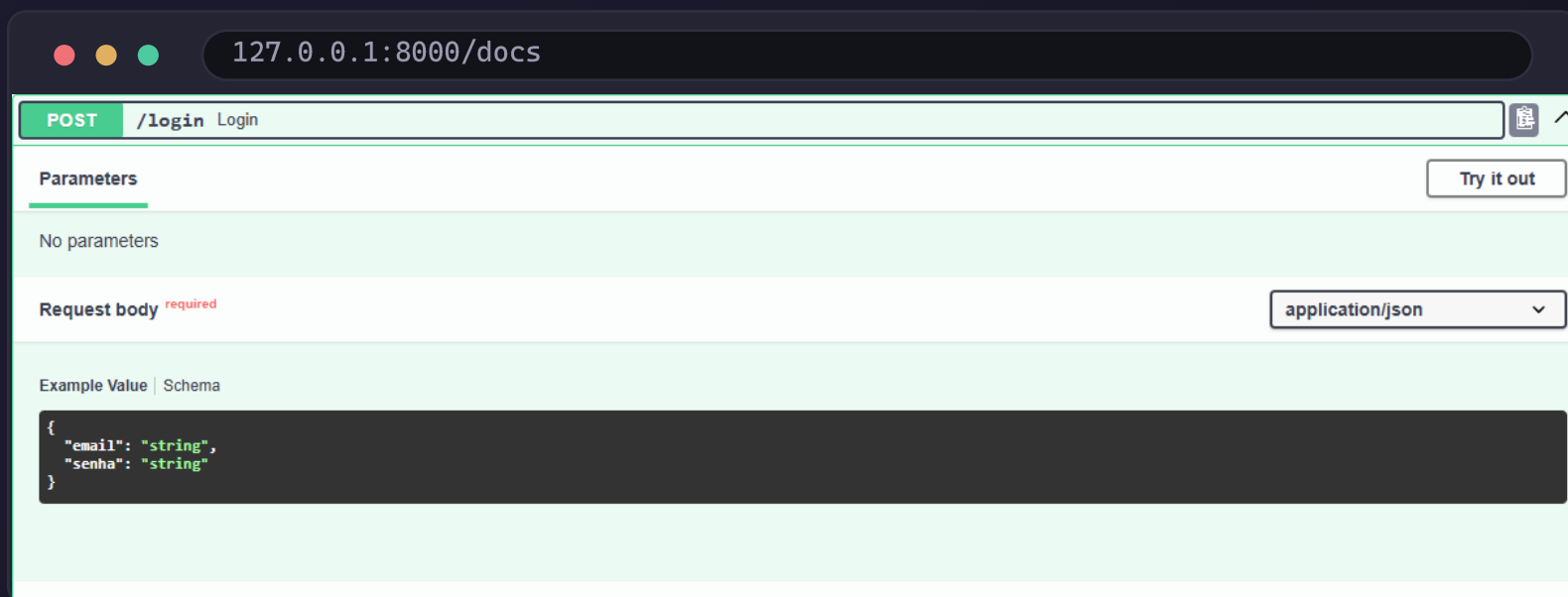
Esta página é o **Swagger**. Ninguém escreveu: o FastAPI montou lendo o `main.py`.

No topo, o **title** da app. Embaixo, a nossa única rota, com o verbo em verde.

Em Schemas está o **DadosLogin** — a classe virou documentação.

Clique na faixa verde **POST /login** para abrir a rota.

Try it out



A rota aberta mostra o que ela espera: nenhum parâmetro no endereço e um Request body obrigatório (required).

O exemplo com **"string"** saiu da classe DadosLogin: dois campos de texto.

Clique em **Try it out**, à direita: o exemplo vira uma caixa editável.

Preenchendo e executando

The screenshot shows the Swagger UI interface for a **POST /login** endpoint. The browser address bar shows `127.0.0.1:8000/docs`. The endpoint is labeled **POST /login Login**. Under the **Parameters** section, it says "No parameters". The **Request body** section is marked as **required** and contains a JSON object: `{ "email": "user@email.com", "senha": "123456" }`. At the bottom, there is a large blue button labeled **Execute**.

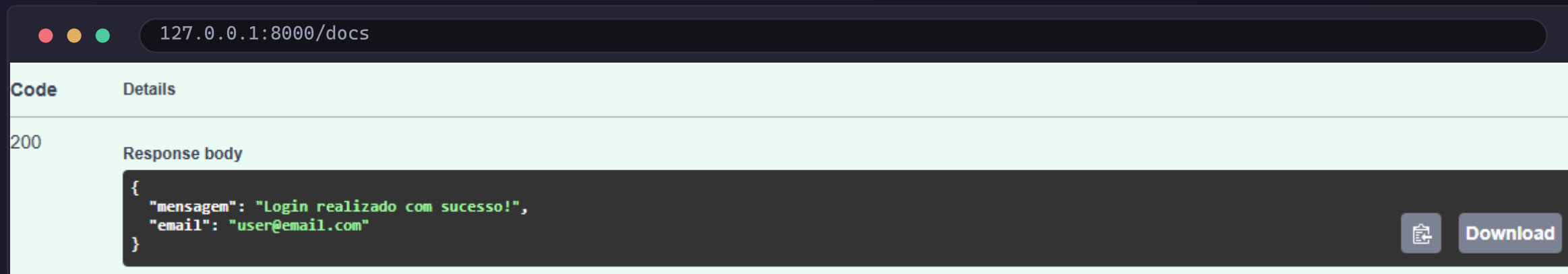
Troque os dois **"string"** pelas credenciais de teste:

user@email.com e **123456**.

Mantenha as aspas duplas e a vírgula entre os campos. Depois, **Execute**.

O Swagger faz o papel do React: monta o pedido POST e manda para a API de verdade.

Senha certa: 200

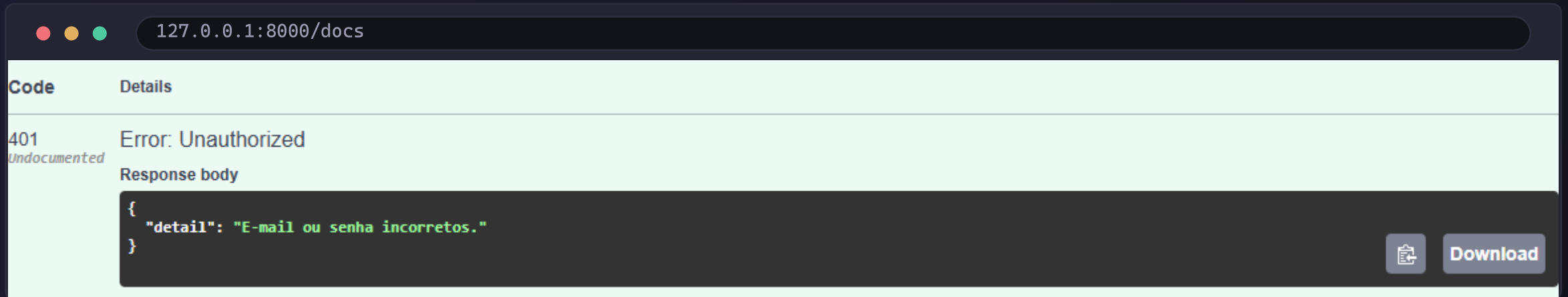


Role até **Server response**. Code **200** e, no Response body, o dicionário do return virado JSON.

Ao mesmo tempo, o terminal registra o pedido numa linha nova: **"POST /login HTTP/1.1" 200 OK**

Acima do resultado, o Swagger mostra o mesmo pedido como comando curl — um jeito de chamar a API sem navegador.

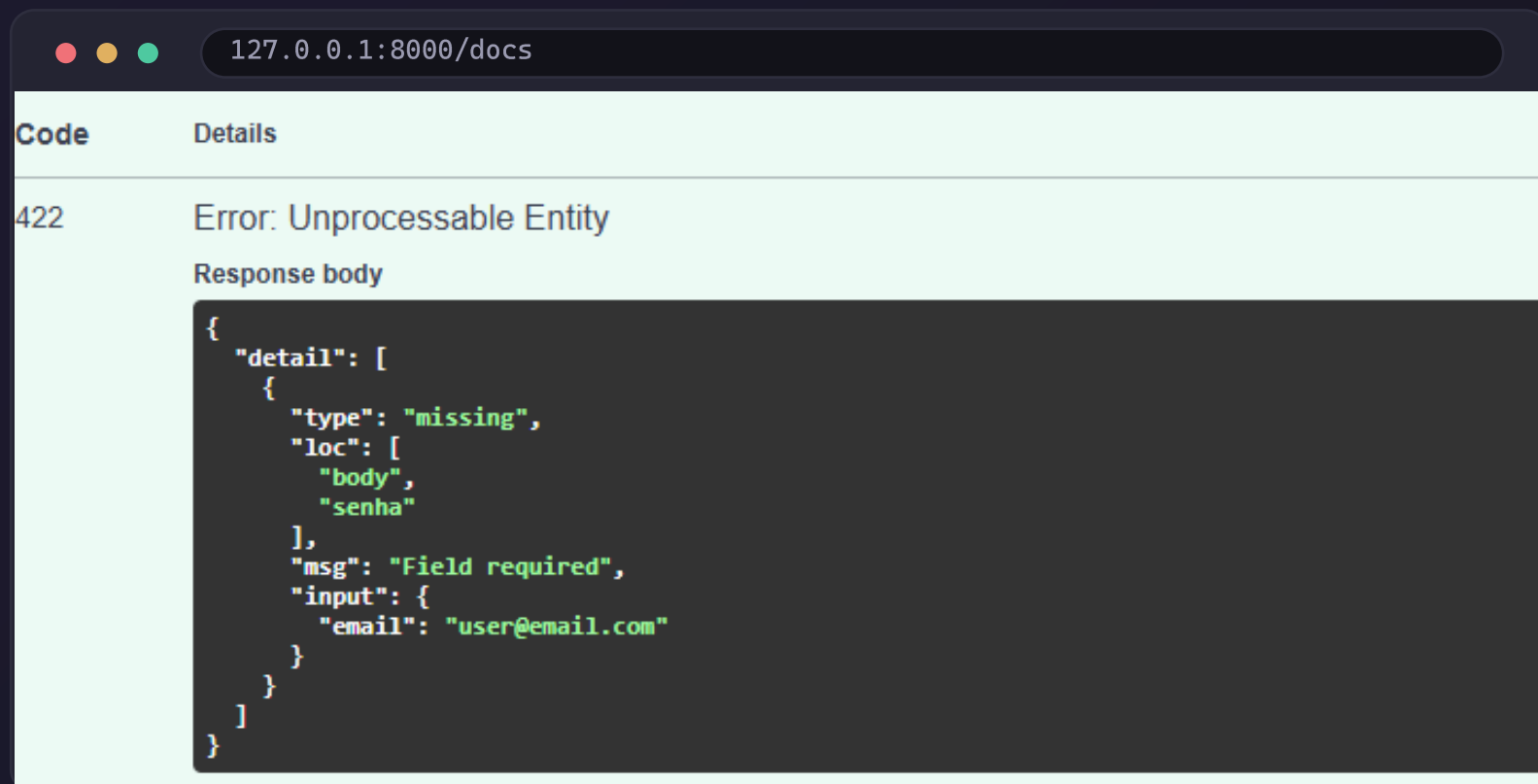
Senha errada: 401



Troque a senha por qualquer outra coisa e execute de novo. Agora a resposta é **401** com o detail do HTTPException: foi o raise que respondeu.

É **esta** resposta que o Login.jsx vai receber para mostrar a mensagem de erro na tela.

Campo faltando: 422



```
127.0.0.1:8000/docs

Code  Details
422    Error: Unprocessable Entity
      Response body
      {
        "detail": [
          {
            "type": "missing",
            "loc": [
              "body",
              "senha"
            ],
            "msg": "Field required",
            "input": {
              "email": "user@email.com"
            }
          }
        ]
      }
    }
```

Apague a linha da senha (e a vírgula que sobrou) e execute.

422: nosso código nem rodou. O Pydantic viu que faltava **senha** no **body** e recusou — é o "Field required".

Essa validação veio de graça, só por termos escrito a classe.

Bônus:

<http://127.0.0.1:8000/redoc>

mostra a mesma documentação em outro formato, só de leitura.

Erros no terminal

- 01** **O termo 'uv' não é reconhecido...** — feche o VS Code inteiro e abra de novo. Persistindo, reinstale o uv.
- 02** **...Activate.ps1 não pode ser carregado...** — rode **Set-ExecutionPolicy -Scope CurrentUser RemoteSigned** e responda S.
- 03** **O termo 'fastapi' não é reconhecido...** — falta o (api-login) na linha: ative o venv, ou rode **uv run fastapi dev main.py**.
- 04** **Path does not exist main.py** — você está fora da pasta. Rode **cd api-login** e tente de novo.

Erros ao testar

- 01** **O navegador não abre /docs** — o terminal do fastapi dev foi fechado ou o Ctrl+C desligou o servidor. Suba de novo.
- 02** **`{"detail": "Not Found"}`** em 127.0.0.1:8000 — é normal: a raiz / não é rota nossa. Acrescente /docs.
- 03** **422 com a senha certa** — o JSON está mal escrito: aspas simples, vírgula faltando entre os campos ou sobrando no último.
- 04** **Salvei o main.py e nada mudou** — aperte F5 na página do /docs: o Swagger não recarrega sozinho.

Erros no main.py

- 01 200 com **null** para qualquer senha — um trecho foi digitado duas vezes (dois def login). Compare com o slide 30.
- 02 **IndentationError** no terminal — espaços errados no começo da linha. O terminal diz o número da linha.
- 03 **SyntaxError: expected ':'** — faltou os dois-pontos no fim do **class**, do **def** ou do **if**.
- 04 500 e **NameError: name 'HTTPException' is not defined** — faltou ele no import da primeira linha.

Voltando ao projeto

```
Terminal · PowerShell

1 # no dia seguinte: abrir a pasta, ativar e subir
2 PS C:\projetos> cd api-login
3 PS C:\projetos\api-login> .venv\Scripts\activate
4 (api-login) PS C:\projetos\api-login> fastapi dev main.py
5
6 # projeto baixado do GitHub, sem a pasta .venv: o uv recria tudo
7 PS C:\projetos\api-login> uv sync
```

Instalar o uv, **uv init**, **uv venv** e **uv add** foram uma vez só. Do segundo dia em diante são três comandos: entrar, ativar e subir.

Clonou o projeto em outro computador? A `.venv` não veio junto — **uv sync** lê o `uv.lock` e recria o ambiente idêntico.

Porta 8000 ocupada por outro servidor esquecido?
Ctrl+C no terminal dele, ou suba em outra: **fastapi dev main.py --port 8001**

Do zero à API, em quatro passos

01

Projeto e ambiente: `uv init api-login --python 3.12` e `uv venv`.

02

Ativar com `.venv\Scripts\activate` e adicionar o `uv add "fastapi[standard]==0.141.1"`.

03

No main.py: `app`, `DadosLogin` e a rota `@app.post("/login")`.

04

Subir com `fastapi dev main.py` e testar 200, 401 e 422 em `/docs`.

Um primeiro passo, não o fim

A senha saiu do navegador — mas **ainda mora no código.**

O e-mail e a senha fixos no main.py são um exercício. Num sistema de verdade os usuários ficam num banco de dados, e a senha é guardada embaralhada (hash), nunca como texto.

A API também ainda não entrega nada que prove o login para os próximos pedidos. Isso é o token, que vai substituir o "true" do localStorage.

E o React ainda confere a senha sozinho: a próxima peça é o Login.jsx perguntando para esta API.

Hoje
POST /login

+

Próximo
o React chama a API

=

Depois

banco + token

0 teu trabalho

01

Refazer a API **do zero** em casa, numa pasta nova, seguindo só os slides.

02

Trocar o **title** da app pelo teu nome e conferir que ele aparece no Swagger.

03

Testar no /docs os três casos — **200**, **401** e **422** — e tirar um print de cada.

04

Entregar os três prints e o **main.py**. Sem a pasta .venv!